

Export and Import Excel Data in C# using the NPOI Library

In this article, I am going to discuss **How to Export and Import Excel Data in C#** with Examples using the NPOI Library. Please read our previous article, where we discussed the [DirectoryInfo Class in C#](#) with Examples. Whenever we are working on Real-Time Projects, it is a common requirement to Import data from an Excel File as well as to Export Data to an Excel File. So, here, first, we will discuss how to Export Excel Data, and then we will see How to Import Excel Data using NPOI Library.

What is NPOI?

NPOI is an Open-Source library that can help us perform read and write operations on XLS, DOC, and PPT file extensions. This NPOI Library is the .NET version of the POI Java project at <http://poi.apache.org/>. This NPOI Library covers almost all the important features of Excel, like styling, formatting, data formulas, extracting images, etc. And the most important thing is that it does not require Microsoft Office to be installed on the server. So, here, in this article, I will show you how to perform the Read and Write Operations on an Excel File using the NPOI Library.

Example to Understand Export Excel Data in C#:

Let us see an Example to Understand How to Export Excel Data in C# using the NPOI Library. So, we want to represent the Employee information of an organization in the following Excel Format.

A	B	C	D	E	F	G	H	I	J
					Dot Net Tutorials Online Training		Company Name		
					1988/2019, 5th floor, Tower B, Bajrang Vihar, Patia, Bhubaneswar-400051, India				
					Company Address		Headers		
SR NO	ID	Name	Address	Email	IsPermanent	Mobile	RegdNo	Salary	ProfileURL
91	190	Name-90	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19224	10090.00	10090
92	191	Name-91	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19225	10091.00	10091
93	192	Name-92	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19226	10092.00	10092
94	193	Name-93	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19227	10093.00	10093
95	194	Name-94	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19228	10094.00	10094
96	195	Name-95	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19229	10095.00	10095
97	196	Name-96	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19230	10096.00	10096
98	197	Name-97	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19231	10097.00	10097
99	198	Name-98	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19232	10098.00	10098
100	199	Name-99	Some Address	meEmail@dotnnettutorials.i	TRUE	0123456789	19233	10099.00	10099
TOTAL							Calculated Column	1004950	

As you can see in the above image, our Excel File is going to contain the following information.

On the Left-Hand Side top corner, we want to put the logo of the Company. On the right-hand side, we want to put the Company Name and Company Address. Here, we need to apply different Cell Styles for Company Name and Address.

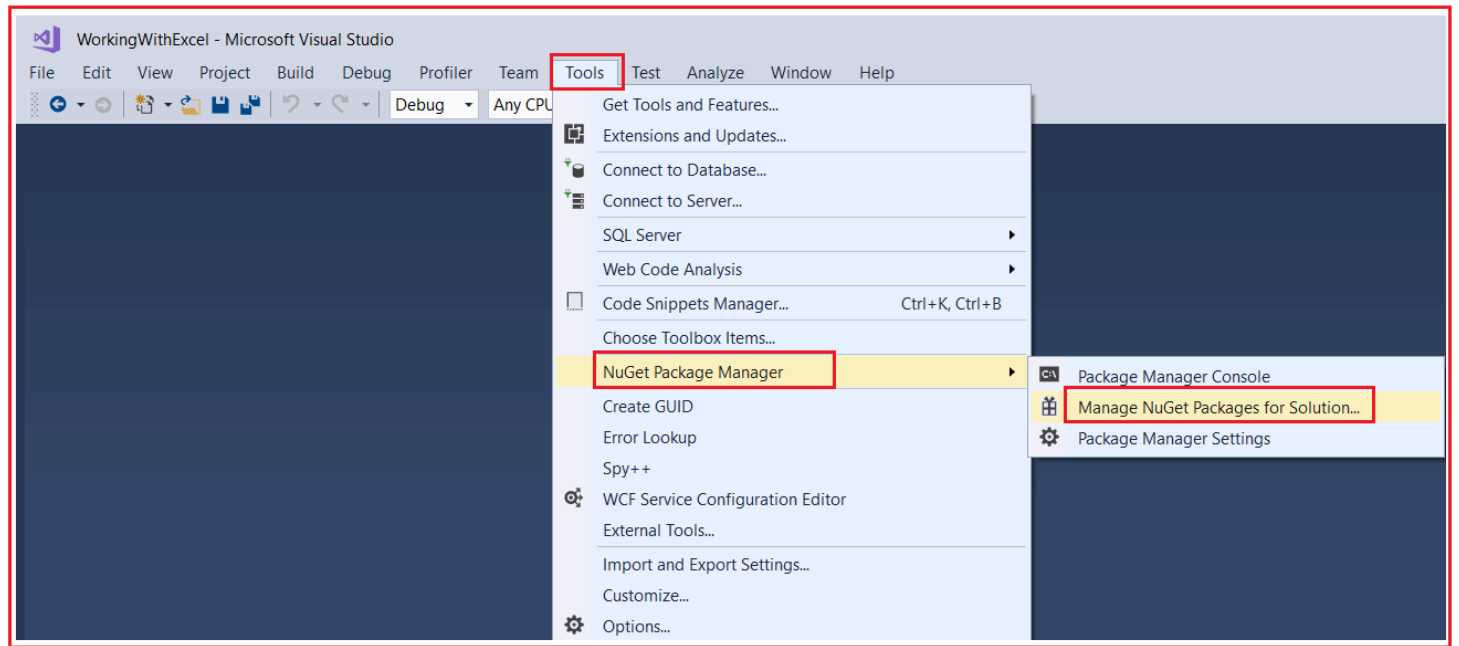
Then, we need to provide the column headers, and for this, we are going to use different Cell Styles. After the Column Header, we have Data Rows, and for Data Rows, we have different cell styles. We have a different cell style for string data and another cell style for numeric data. Further, if you observe the ProfileURL column, we need to make that column value a Hyperlink, and on clicking on that hyperlink, it should open the appropriate URL on your browser. For this hyperlink column data, we have a different cell style.

Once the Data Rows are completed, we need to calculate the total salary. For this, we need to use Excel Formula, and for this, we are using a different cell style. The formula will be to sum all the column values of the Salary column and show the result on the Total Row.

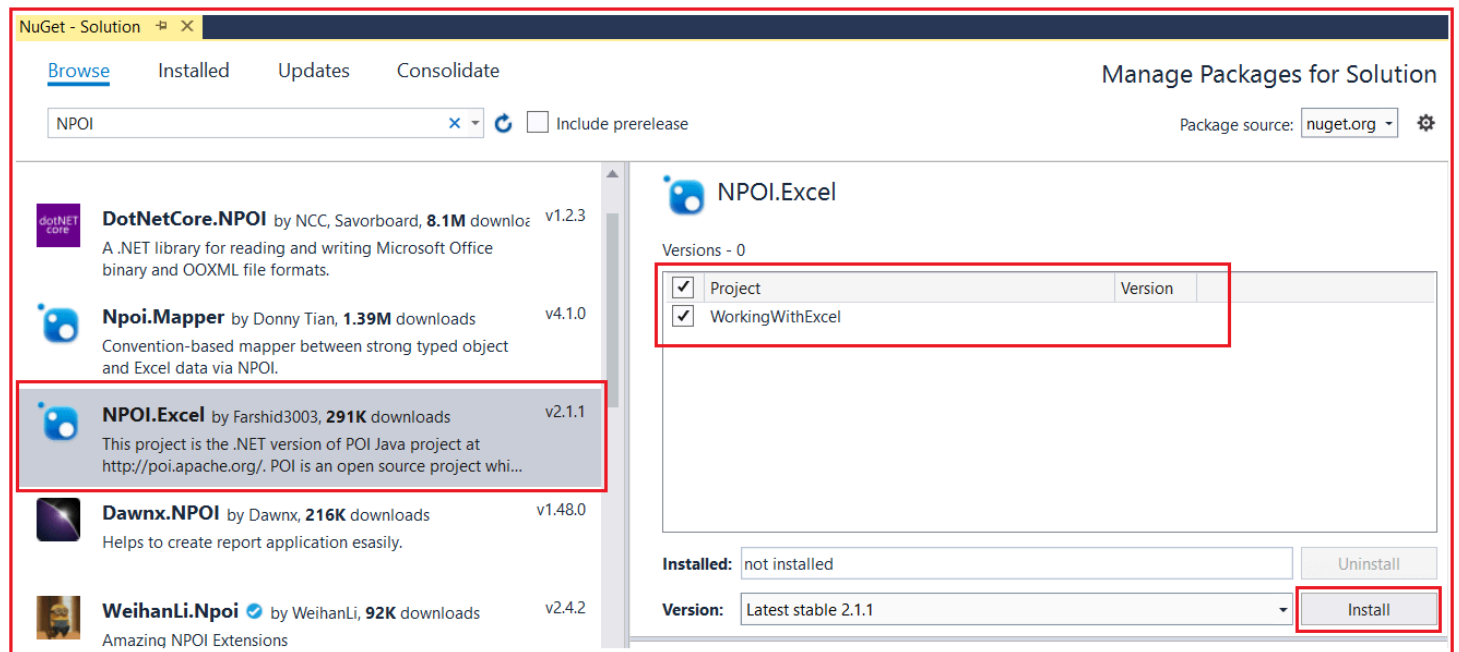
Here, we also need to freeze the company logo, company name, company address, and headers while scrolling down to see the data rows. Let us proceed and see how to implement the above-discussed example in C# using the NPOI Library.

Export Excel Data Example in C# using NPOI Library:

So, create a Console application with the name WorkingWithExcel. Once you create the Project, then you need to install the NPOI Library from NuGet Package Manager. To do so, Click on **Tools => NuGet Package Manager => Manage NuGet Packages for Solution**, as shown in the below image.



Then go to the Browse tab, search for NPOI, and select the **NPOI.Excel** Library, select the project where you want to install the Library, and finally click on the Install button as shown in the below image.



Once you have Installed the NPOI Library, create a folder with the name Images, and inside this folder, please download and save the following image.



Next, create another folder with the name ExcelFiles and inside this folder, our generated Excel Files will be stored. Next, create a class file with the name **Employee.cs** and then copy and paste the following code. This is going to be our model class, which is going to hold the Employee data that we want to export into the Excel File.

```
namespace WorkingWithExcel
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Address { get; set; }
        public string Email { get; set; }
        public string Mobile { get; set; }
        public bool IsPermanent { get; set; }
        public int RegdNo { get; set; }
        public int Salary { get; set; }
        public string ProfileURL { get; set; }
    }
}
```

Next, modify the Program class as follows. In the code below, we use the NPOI Library to Export the Data into an Excel file. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.Collections.Generic;
using NPOI.HSSF.UserModel;
using NPOI.HSSF.Util;
using NPOI.SS.Util;
using NPOI.SS.UserModel;
using System.IO;
namespace WorkingWithExcel
{
    public class Program
    {
        public static void Main()
        {
            try
            {
                #region Data
                //In Real-Time you will get the data from the database,
                //but here we have hard-coded the data
```

```

List<Employee> empList = new List<Employee>();
for (int i = 0; i < 100; i++)
{
    Employee emp = new Employee()
    {
        ID = 100 + i,
        Name = "Name-" + i,
        Address = "Some Address",
        Email = "SameEmail@dotnettutorials.net",
        IsPermanent = true,
        Mobile = "0123456789",
        RegdNo = 12345 + i + 6789,
        Salary = 10000 + i,
        ProfileURL = "100" + i
    };
    empList.Add(emp);
}
#endregion
#region Creating Excel Sheet
//The following Pieces of Code will Create the Excel File,
//Excel Sheet and font object which will be used later
HSSFWorkbook workbook = new HSSFWorkbook();
//The sheet name is going to be Dot Net Tutorials
HSSFSheet sheet = (HSSFSheet)workbook.CreateSheet("Dot Net Tutorials");
HSSFFont font = (HSSFFont)workbook.CreateFont();
#endregion
#region Creating Different Cell Styles
//Now, we going to create different cell styles for different data
#region Company Cell Styles
//We will use the following cell style with the Company Name
var Company = workbook.CreateCellStyle();
Company.Alignment = HorizontalAlignment.Left;
var CompanyFont = workbook.CreateFont();
CompanyFont.FontName = "Arial";
CompanyFont.Color = HSSFColor.Blue.Index;
CompanyFont.Boldweight = (short)FontBoldWeight.Bold;
CompanyFont.FontHeightInPoints = ((short)16);
Company.SetFont(CompanyFont);
#endregion
#region Address Cell Style
//We will use the following cell style with the Company Address
var Address = workbook.CreateCellStyle();
Address.Alignment = HorizontalAlignment.Left;
var AddressFont = workbook.CreateFont();
AddressFont.FontName = "Arial";
AddressFont.Boldweight = (short)FontBoldWeight.Bold;
AddressFont.FontHeightInPoints = ((short)10);
Address.SetFont(AddressFont);
#endregion
#region Header Cell Style
//We will use the following cell style with the Header

```

```

var Header = workbook.CreateCellStyle();
Header.Alignment = HorizontalAlignment.Center;
Header.FillForegroundColor = NPOI.HSSF.Util.HSSFColor.LightBlue.Index;
Header.FillBackgroundColor = NPOI.HSSF.Util.HSSFColor.LightBlue.Index;
Header.FillPattern = FillPattern.SolidForeground;
var HeaderFont = workbook.CreateFont();
HeaderFont.FontName = "Arial";
HeaderFont.Boldweight = (short)FontBoldWeight.Bold;
HeaderFont.Color = HSSFColor.White.Index;
HeaderFont.FontHeightInPoints = ((short)10);
Header.SetFont(HeaderFont);
Header.BorderLeft = BorderStyle.Thin;
Header.BorderTop = BorderStyle.Thin;
Header.BorderRight = BorderStyle.Thin;
Header.BorderBottom = BorderStyle.Thin;
#endregion
#region Number Data Cell Style
//We will use the following cell style with Data which is Numeric
var NumData = workbook.CreateCellStyle();
var formatId = HSSFFormat.GetBuiltinFormat("##0.00");
if (formatId == -1)
{
var newDataFormat = workbook.CreateDataFormat();
NumData.DataFormat = newDataFormat.GetFormat("##0.00");
}
else
NumData.DataFormat = formatId;
#endregion
#region Data Cell Style
//We will use the following cell style with Data that is NOT Numeric
var Data = workbook.CreateCellStyle();
Data.Alignment = HorizontalAlignment.Center;
var DataFont = workbook.CreateFont();
DataFont.FontName = "Arial";
DataFont.FontHeightInPoints = ((short)9);
Data.SetFont(DataFont);
Data.BorderLeft = BorderStyle.Thin;
Data.BorderTop = BorderStyle.Thin;
Data.BorderRight = BorderStyle.Thin;
Data.BorderBottom = BorderStyle.Thin;
#endregion
#region Link Data Cell Style
//We will use the following cell style with Hyperlink Data
var linkData = workbook.CreateCellStyle();
linkData.Alignment = HorizontalAlignment.Center;
var linkDataFont = workbook.CreateFont();
linkDataFont.FontName = "Arial";
linkDataFont.Color = HSSFColor.Blue.Index;
linkDataFont.FontHeightInPoints = ((short)9);
linkDataFont.Underline = FontUnderlineType.Single;
linkDataFont.Color = HSSFColor.Blue.Index;

```

```

linkData.SetFont(linkDataFont);
linkData.BorderLeft = BorderStyle.Thin;
linkData.BorderTop = BorderStyle.Thin;
linkData.BorderRight = BorderStyle.Thin;
linkData.BorderBottom = BorderStyle.Thin;
#endregion
#endregion
#region Creating Company and Address of the Excel Sheet
//Creating the Company Name from 2nd Row by applying Company Cell Style
// rowIndex is going to hold the Row Number. Inex means 0 - Based Index
int rowIndex = 2; //rowIndex 2 means 3rd Row
var row = sheet.CreateRow(rowIndex);
var cell = row.CreateCell(4);
cell.SetCellValue("Dot Net Tutorials Online Training");
cell.CellStyle = Company;
sheet.AddMergedRegion(new CellRangeAddress(4, 4, 4, 14));
//Creating the Company Address from 3rd Row by applying Address Cell Style
rowIndex = rowIndex + 1;
var row1 = sheet.CreateRow(rowIndex);
var cell1 = row1.CreateCell(4);
cell1.SetCellValue("1988/2019, 5th floor, Tower B, Bajrang Vihar, Patia, Bhuba
India");
cell1.CellStyle = Address;
sheet.AddMergedRegion(new CellRangeAddress(5, 5, 4, 14));
#endregion
// Set Row index for Header
rowIndex = 7; //rowIndex 7 means 8th Row which is going to be our Header in Ex
var SR_NO = 0; //We want a unique Serial Number for Each Row in the Excel Shee
#region Excel Data Headers
var cellheaderindex = 0;
var excelheaderrow = sheet.CreateRow(rowIndex);
var excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("SR NO");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("ID");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("Name");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("Address");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("Email");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;

```

```

excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("IsPermanent");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("Mobile");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("RegdNo");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("Salary");
excelheadercell.CellStyle = Header;
cellheaderindex = cellheaderindex + 1;
excelheadercell = excelheaderrow.CreateCell(cellheaderindex);
excelheadercell.SetCellValue("ProfileURL");
excelheadercell.CellStyle = Header;
#endregion
#region Excel Data
foreach (Employee data in empList)
{
//Increase the rowIndex and SR_NO by 1 for each record in the empList
rowIndex = rowIndex + 1; //This will be the row number in the Excel Sheet
SR_NO = SR_NO + 1; //Unique Serial Number
var cellindex = 0; //Cell Number starting from 0
//Create the New Row
var gridrow = sheet.CreateRow(rowIndex);
//Create the first Cell in the new Row
var gridcell = gridrow.CreateCell(cellindex);
//Add value to the Cell
gridcell.SetCellValue(SR_NO);
//Apply appropriate CSS Styles
gridcell.CellStyle = Data;
//Increase the Cell Index by 1 to create the next cell in the Row
cellindex = cellindex + 1;
//Create the new cell
gridcell = gridrow.CreateCell(cellindex);
//Add value to the Cell
gridcell.SetCellValue(data.ID);
//Apply appropriate CSS Styles
gridcell.CellStyle = Data;
//The Process will continue till the last cell in the Row
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.Name);
gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.Address);

```

```

gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.Email);
gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.IsPermanent);
gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.Mobile);
gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.RegdNo);
gridcell.CellStyle = Data;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(Convert.ToDouble(data.Salary));
gridcell.CellStyle = NumData;
cellindex = cellindex + 1;
gridcell = gridrow.CreateCell(cellindex);
gridcell.SetCellValue(data.ProfileURL);
//Setting the Cell value as Hyper link
HSSFHyperlink link = new HSSFHyperlink(HyperlinkType.Url)
{
    //On click on the Hyperlink, it will open the following URL in the browser
    Address = "http://www.dotnettutorials.net/" + data.ProfileURL
};
gridcell.Hyperlink = (link);
gridcell.CellStyle = linkData;
}
#endregion
#region Freezing Point
//Setting the Freezing Point- Column 0 and Row Number 8
sheet.CreateFreezePane(0, 8, 0, 8);
for (int i = 0; i <= cellheaderindex; i++)
{
    sheet.SetColumnWidth(i, 5000);
}
#endregion
#region TOTAL & FORMULA SECTION
//We need to calculate the sum of the salary column
var startrow = 9; //Data Rows started from Row Number 9
var lastdatarow = rowIndex + 1; //Last Row
rowIndex = rowIndex + 2; //Creating a New Rows to display the Total
var Formularow = sheet.CreateRow(rowIndex);
var Formulacell = Formularow.CreateCell(0);
Formulacell.SetCellValue("TOTAL");
Formulacell.CellStyle = Header;

```

```

//Creating a Cell to display the Total
Formulacell = Formularow.CreateCell(8);
//Formula to calculate the Total
Formulacell.CellStyle = Header;
String strFormula = "SUBTOTAL(9,I" + Convert.ToString(startrow) + ":I" +
Convert.ToString(lastdatarow) + ")";
Formulacell.SetCellType(CellType.Formula);
Formulacell.SetCellFormula(strFormula);
HSSFFormulaEvaluator.EvaluateAllFormulaCells(workbook);
#endregion
#region LOGO
//Displaying the Logo on the TOP Left Corner of the Excel Sheet
HSSFPatriarch patriarch = (HSSFPatriarch)sheet.CreateDrawingPatriarch();
HSSFClientAnchor anchor = new HSSFClientAnchor(0, 0, 0, 255, 1, 2, 2, 5)
{
AnchorType = (int)NPOI.SS.UserModel.AnchorType.MoveAndResize
};
//Here, you need to replace the Image Path and Name as per your directory str
HSSFPicture picture = (HSSFPicture)patriarch.CreatePicture(anchor,
LoadImage(@"C:\Users\HP\source\repos\WorkingWithExcel\WorkingWithExcel\Images\
workbook));
picture.Resize();
picture.LineStyle = (LineStyle)HSSFPicture.LINESTYLE_NONE;
#endregion
//Finally, Create the Excel File and Save it on a specified Location
string FileName = "MyExcel_" + DateTime.Now.ToString("yyyy-dd-MM--HH-mm-ss") +
//Here, you need to replace the Path as per your directory structure where you
image
using (FileStream file = new
FileStream(@"C:\Users\HP\source\repos\WorkingWithExcel\WorkingWithExcel\ExcelF
FileMode.Create))
{
workbook.Write(file);
file.Close();
Console.WriteLine("File Created Successfully...");
}
}
catch (Exception ex)
{
Console.WriteLine(ex.Message);
}
Console.ReadKey();
}
//The following code is used to load the Image, Create the Image and return th
public static int LoadImage(string path, HSSFWorkbook wb)
{
FileStream file = new FileStream(path, FileMode.Open, FileAccess.Read);
byte[] buffer = new byte[file.Length];
file.Read(buffer, 0, (int)file.Length);
return wb.AddPicture(buffer, PictureType.JPEG);
}

```

```
}  
}
```

With the above changes in place, now run the application code and see the output. If everything is fine then you will get the message saying **File Created Successfully....** Now, go to the Excel Files folder and you will see the Excel should be generated with the required format, formula, and data.

Note: Styling the Header, Data, etc. are optional. If you don't want to, you can remove them.

Import Excel Sheet Data in C# using NPOI Library:

Now, let us proceed and understand how to Import Excel Data in C# using the NPOI Library. Let us assume we have the following MyExcelFile.xlsx file in our Excel Files folder.

A	B	C	D	E	F	G	H
ID	Name	Address	Email	IsPermanent	Mobile	RegdNo	Salary
100	Name-0	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19134	10000.00
101	Name-1	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19135	10001.00
102	Name-2	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19136	10002.00
103	Name-3	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19137	10003.00
104	Name-4	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19138	10004.00
105	Name-5	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19139	10005.00
106	Name-6	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19140	10006.00
107	Name-7	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19141	10007.00
108	Name-8	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19142	10008.00
109	Name-9	Some Address	SameEmail@dotnettutorials.net	TRUE	0123456789	19143	10009.00

As you can see in the above image, this is nothing but the employee data of an organization. Now, we need to import the above Excel Sheet into our .NET Application. So, for this, please modify the Program class as follows. The following code is self-explained, so please go through the comment lines for a better understanding.

```
using System;  
using System.Collections.Generic;  
using NPOI.SS.UserModel;  
using System.IO;  
using NPOI.XSSF.UserModel;  
namespace WorkingWithExcel  
{  
    public class Program  
    {  
        public static void Main()  
        {  
            try  
            {  
                //The following Collection object is going to hold the Excel Sheet Data  
                List<Employee> listEmployees = new List<Employee>();  
                //Set the Excel File Path
```

```

string FilePath =
@"C:\Users\HP\source\repos\WorkingWithExcel\WorkingWithExcel\ExcelFiles\MyExcelFile.xlsx";
//For Import we need to create an instance of XSSFWorkbook class
XSSFWorkbook xssfWorkbook;
using (FileStream file = new FileStream(FilePath, FileMode.Open, FileAccess.Read))
{
    xssfWorkbook = new XSSFWorkbook(file);
}
//The Excel File Might have multiple sheets
//So create an ISheet object by specifying the appropriate sheet name or index
//ISheet sheet = xssfWorkbook.GetSheet("Sheet1"); //Based on the Sheet Name
ISheet sheet = xssfWorkbook.GetSheetAt(0); //Based on the Index Position (0-Based Index)
//If First Row is Header, then Initialize the row = 1 else row = 0 inside the loop
//As our Excel Sheet Contains the First Row as a Header, we are setting row = 1
//The following Loop will start from the First data Rows till the last data row
for (int row = 1; row <= sheet.LastRowNum; row++)
{
    //null is when the row only contains empty cells
    if (sheet.GetRow(row) != null)
    {
        //If the Row is not empty, then Fetch cell value and populate them with Employee object
        Employee emp = new Employee();
        //For Accessing Numeric Cell Value we need to use NumericCellValue Property
        //For Accessing String Cell Value we need to use StringCellValue Property
        //For Accessing Boolean Cell Value we need to use BooleanCellValue Property
        //First Cell is ID
        ID = (int)sheet.GetRow(row).GetCell(0).NumericCellValue,
        //Second Cell is Name
        Name = sheet.GetRow(row).GetCell(1).StringCellValue,
        //Third Cell is Address
        Address = sheet.GetRow(row).GetCell(2).StringCellValue,
        //Fourth Cell is Email
        Email = sheet.GetRow(row).GetCell(3).StringCellValue,
        //Fifth Cell is IsPermanent
        IsPermanent = sheet.GetRow(row).GetCell(4).BooleanCellValue,
        //Sixth Cell is Mobile
        Mobile = sheet.GetRow(row).GetCell(5).StringCellValue,
        //Seventh Cell is RegdNo
        RegdNo = (int)sheet.GetRow(row).GetCell(6).NumericCellValue,
        //Eighth Cell is Salary
        Salary = (int)sheet.GetRow(row).GetCell(7).NumericCellValue,
    };
    //Add the employee data into listEmployees Collection
    listEmployees.Add(emp);
}
}
//Once you import the data from the Excel File to your .NET Object, then you can do whatever you want

```

```
//Like Save the Data into the Database, Display the Data, Modify the Data, etc
//Here, we are going to Display the Excel Sheet Data
foreach (var emp in listEmployees)
{
    Console.WriteLine($"ID: {emp.ID}, Name: {emp.Name}, Address: {emp.Address}, Salary: {emp.Salary}");
}
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
Console.ReadKey();
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Address { get; set; }
    public string Email { get; set; }
    public string Mobile { get; set; }
    public bool IsPermanent { get; set; }
    public int RegdNo { get; set; }
    public int Salary { get; set; }
}
}
```

Output:

```
ID: 100, Name: Name-0, Address: Some Address, Salary: 10000
ID: 101, Name: Name-1, Address: Some Address, Salary: 10001
ID: 102, Name: Name-2, Address: Some Address, Salary: 10002
ID: 103, Name: Name-3, Address: Some Address, Salary: 10003
ID: 104, Name: Name-4, Address: Some Address, Salary: 10004
ID: 105, Name: Name-5, Address: Some Address, Salary: 10005
ID: 106, Name: Name-6, Address: Some Address, Salary: 10006
ID: 107, Name: Name-7, Address: Some Address, Salary: 10007
ID: 108, Name: Name-8, Address: Some Address, Salary: 10008
ID: 109, Name: Name-9, Address: Some Address, Salary: 10009
```

In the next article, I will discuss **How to generate PDF in C#** with Examples. Here, in this article, I try to explain **How to Perform Export and Import Excel Data in C#** with Examples using the NPOI Library. I hope you enjoy this Export and Import Excel Data in C# with Examples article.